ES 248 — End-Semester MCQs

Introduction to DSA in C Programming · 15 questions for review

Each question is a single-answer MCQ unless otherwise noted (Q2 has two parts: time and space complexity). Answer key is on the last page.

Instructions

- All code is in **C**. Assume standard C semantics unless a question specifies otherwise (particular `sizeof` values, endianness, alignment, etc.).
- Choose **one** option per question. **Q2** has two sub-parts (time and space complexity) — answer each separately.
- No negative marking.

Time Complexity

Q1. What is the time complexity of the following code in terms of `n`?

```
for (int i = 1; i <= n; i = i * 2)
    for (int j = 0; j < n; j++)
        printf("*");
```

A) O(n)

B) O(n log n)

C) $O(n^2)$

D) O(log n)

Q2. Consider the following recursive function:

```
void recursion(int N) {
    if (N == 0) return;
    printf("%d\n", N);
    recursion(N - 1);
}
```

Time complexity:

A) O(1)

B) $O(2^N)$

C) O(N)

D) O(log N)

Space complexity:

A) O(1)

B) O(log N)

C) $O(2^N)$

D) O(N)

Q3. What is the time complexity of the following code?

```
void calc(int arr[], int N) {
    int j = 0;
    for (int i = 0; i < N; i++) {
        while (j < N && arr[i] < arr[j]) {
            j++;
        }
    }
}
```

A) O(log N)

B) O(N)

C) O(N · log N)

D) $O(N^2)$

Linked Lists

Q4. A singly linked list contains the nodes `A → B → C → D → NULL`, and `head` is a pointer to node A. After executing `head->next = head->next->next;`, which statement is correct?

A) The list becomes `A → C → D → NULL`; node B is leaked

B) The list becomes `A → C → D → NULL`; no memory leak

C) The list becomes `B → C → D → NULL`; node A is leaked

D) The program crashes with a segmentation fault

Q5. A singly linked list maintains both `head` and `tail` pointers. Which of the following operations is **NOT** O(1)?

A) Insert at head

B) Insert at tail

C) Delete from head

D) Delete from tail

Q6. Consider the following function intended to free every node of a singly linked list:

```
void destroyList(struct Node *head) {
    while (head != NULL) {
        free(head);
        head = head->next;
    }
}
```

What is wrong with this code?

A) The base case is incorrect; should be `head == NULL`

B) After `free(head)`, accessing `head->next` reads freed memory — undefined behavior

C) The function will crash with a segmentation fault if `head` is `NULL` when called

D) The function correctly frees all nodes

Queues

Q7. A queue is implemented as a circular array of capacity 8, with fields `front`, `rear` (next insertion index), and `size`. Currently `front = 6`, `rear = 2`, `size = 4`. Which array indices currently hold queue elements?

A) 2, 3, 4, 5

B) 6, 7, 0, 1

C) 6, 7, 8, 0

D) 0, 1, 6, 7

Q8. Items 1, 2, 3, 4, 5 are pushed onto a stack in that order. They are then popped one at a time and each popped item is immediately enqueued onto a queue. After all five items are transferred, the queue is fully dequeued. In what order are the items dequeued?

A) 1, 2, 3, 4, 5

B) 5, 4, 3, 2, 1

C) 1, 3, 5, 4, 2

D) 5, 1, 4, 2, 3

Function Pointers & qsort

Q9. What does the following declaration declare?

```
int (*funcs[10])(int, int);
```

A) An array of 10 functions, each returning an `int` and taking two `int` parameters

B) An array of 10 pointers to functions, each taking two `int`s and returning an `int`

C) A pointer to an array of 10 functions

D) A function returning an array of 10 `int`s

Q10. Which `qsort` comparator sorts an array of `struct Student` by `score` in **descending** order, with ties broken alphabetically by `name`?

```
struct Student { char name[100]; int score; };
```

A)

```
int cmp(const void *a, const void *b) {
    return ((struct Student*)b)->score - ((struct Student*)a)->score;
}
```

B)

```
int cmp(const void *a, const void *b) {
    struct Student *sa = (struct Student*)a;
    struct Student *sb = (struct Student*)b;
    if (sa->score != sb->score)
        return sb->score - sa->score;
    return strcmp(sa->name, sb->name);
}
```

C)

```
int cmp(const void *a, const void *b) {
    struct Student *sa = (struct Student*)a;
    struct Student *sb = (struct Student*)b;
    if (sa->score != sb->score)
        return sa->score - sb->score;
    return strcmp(sa->name, sb->name);
}
```

D)

```
int cmp(const void *a, const void *b) {
    return ((struct Student*)b)->score - ((struct Student*)a)->score
         + strcmp(((struct Student*)a)->name, ((struct Student*)b)->name);
}
```

Q11. You want to sort the following array of strings alphabetically:

```
char *names[] = {"banana", "apple", "cherry"};
qsort(names, 3, sizeof(char *), cmp);
```

Which definition of `cmp` works correctly?

A) Pass `strcmp` directly as `cmp`

B)
```
int cmp(const void *a, const void *b) {
    return strcmp((char *)a, (char *)b);
}
```

C)
```
int cmp(const void *a, const void *b) {
    return strcmp(*(char **)a, *(char **)b);
}
```

D)
```
int cmp(const void *a, const void *b) {
    return *(char *)a - *(char *)b;
}
```

Structures & Unions

Q12. Assume a hypothetical system where `sizeof(char) = 1`, `sizeof(int) = 4`, alignment is 4 bytes (every member starts at a 4-byte boundary, struct size rounded up to a multiple of 4).

```
struct S {
    char a;
    int  b;
    char c;
};
```

What is `sizeof(struct S)`?

A) 6

B) 9

C) 12

D) 16

Q13. On a **big-endian** system (the most-significant byte is stored at the lowest memory address). Assume `sizeof(int) = 4` and `sizeof(char) = 1`.

```
union U {
    int  i;
    char c[4];
};
union U u;
u.i = 0x12345678;
```

What is the value of `u.c[0]`?

A) 0x12

B) 0x78

C) 0x34

D) 0x56

2D Arrays & Pointers

Q14. Given `int arr[3][4];` and `sizeof(int) = 2`, what is *address of* `arr[2][2]` – *address of* `arr[1][2]` (in bytes)?

A) 4

B) 8

C) 12

D) 16

Q15. Consider:

```
int a[3][4];
int *q       = a[0];
int (*p)[4]  = a;
```

Initially `q` and `p` hold the same address — the address of `a[0][0]`. After executing `q++` and `p++` (assume `sizeof(int) = 2`), by how many bytes have `q` and `p` moved, respectively?

A) 2 bytes, 8 bytes

B) 4 bytes, 16 bytes

C) 8 bytes, 8 bytes

D) 2 bytes, 2 bytes

Answer Key & Rationale

#	Topic	Answer	Rationale / Trick
1	Time	**B**	Outer loop runs $\log_2 n$ times; inner runs n times → O(n log n).
2	Time	**C, D**	N recursive calls (time O(N)); N stack frames (space O(N)). *Trick:* students often answer O(1) for space because no array is allocated.
3	Time	**B**	`j` is initialized outside the outer loop and never reset. It advances at most N times across the entire run → O(N). *Trick:* nested loop shape suggests $O(N^2)$.
4	Linked Lists	**A**	The pointer to node B is overwritten without freeing B → memory leak. List structure is correct, but B is unreachable.
5	Linked Lists	**D**	Even with a tail pointer, deleting the tail node in a *singly* linked list requires traversing from head to find the new tail → O(n).
6	Linked Lists	**B**	After `free(head)`, the memory is gone — reading `head->next` is use-after-free (undefined behavior). Fix: save `next` before freeing. *Trick:* C is plainly false (the `while (head != NULL)` condition handles the empty-list case on the very first line); rules itself out on careful reading.
7	Queues	**B**	Elements occupy `front, front+1, ..., front+size-1` mod 8 = 6, 7, 0, 1.
8	Queues	**B**	Stack reverses order (pop yields 5,4,3,2,1). Queue preserves order (FIFO). Final dequeue order = enqueue order = 5,4,3,2,1.
9	Function Pointers	**B**	Right-left walk: `funcs` → `[10]` (array of 10) → `*` (of pointers) → exit parens → `(int, int)` (to functions taking two ints) → `int` (returning int). *Trick:* A misses that C cannot have arrays of functions, only of pointers to functions.
10	Function Pointers	**B**	Primary key `score` descending (`sb->score - sa->score`); on tie, secondary key `name` ascending via `strcmp`. *Trick:* C reverses the score direction; D combines criteria nonsensically.
11	Function Pointers	**C**	`qsort` passes pointers to **elements**; the elements are `char *`, so `qsort` passes `char **`. The comparator must dereference once before `strcmp`. *Trick:* A is a type mismatch with the qsort signature; B casts `char **` to `char *` → garbage; D compares only the first byte.
12	Structures	**C**	a (1 byte) + 3 bytes **internal padding** + b (4 bytes) + c (1 byte) + 3 bytes **trailing padding** = 12 bytes. *Trick:* forgetting trailing padding gives 9.
13	Unions	**A**	Big-endian places the most-significant byte at the lowest address → `c[0] = 0x12`. *Compile-and-check trap:* a student who runs this on x86 or ARM (both little-endian) gets `0x78` and picks B.
14	2D Arrays	**B**	One row apart = 4 ints × 2 bytes/int = 8 bytes. *Compile-and-check trap:* on a real machine `sizeof(int) = 4`, so a student who runs

			`(char*)&arr[2][2] - (char*)&arr[1][2]` gets 16 and picks D.
15	2D Arrays	**A**	`q` is `int *` → `q++` advances by `1 × sizeof(int) = 2` bytes. `p` is `int (*)[4]` → `p++` advances by `1 × sizeof(int[4]) = 8` bytes. Same starting address; type determines the step. *Compile-and-check trap:* on a real machine `sizeof(int) = 4`, so the student gets 4 and 16 and picks B.